

## Проблема

Эффективная работа с реляционными базами данных требует владения SQL и глубокого понимания схемы данных — это создаёт высокий порог вхождения для большинства пользователей. Решением является перевод произвольного запроса на естественном языке в SQL-запрос автоматически (Text-to-SQL).

Большие языковые модели владеют синтаксисом SQL, однако не обладают знанием конкретной схемы базы данных и ее атрибутов. Результат — модель генерирует синтаксически верный запрос, который падает при выполнении.

## Архитектура

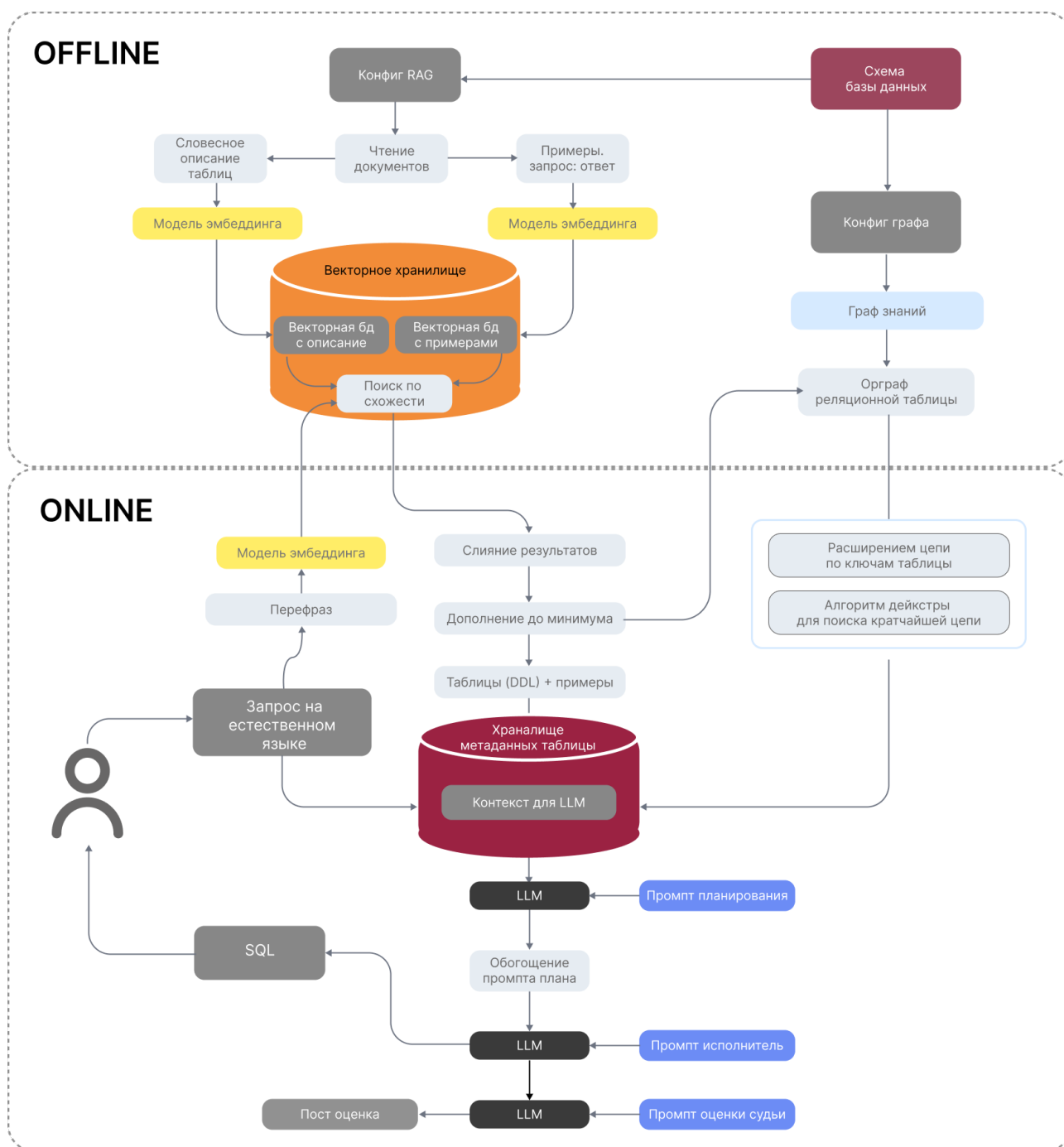


Рис. 1: Схема архитектуры системы

Система работает в два режима. При старте выполняется офлайн-подготовка: схема индексируется в два векторных индекса и строится граф внешних ключей. При каждом запросе запускается онлайн-пайплайн из пяти шагов: перефраз, семантический ретривал, структурное обогащение через граф, планирование JOIN-цепочки и генерация SQL.

## Реализация

**Перефраз запроса.** Перед ретривалом вопрос пользователя перефразируется моделью — из разговорной формулировки в более формальную, терминологически близкую к описаниям схемы. Это повышает семантическое сходство между запросом и retrieval-текстами таблиц и улучшает точность поиска без изменения смысла вопроса.

**Двухиндексный семантический ретривал.** Вместо одного индекса используются два отдельных — для описаний таблиц и для примеров запросов. Разделение принципиально: найденный пример почти напрямую задаёт паттерн JOIN и агрегации, тогда как описание таблицы даёт структурный контекст. Смешение этих сигналов в одном индексе приводит к вытеснению одного типа информации другим. Результаты фильтруются по порогу сходства, нормализуются относительно максимума внутри каждого индекса и объединяются взвешенным слиянием. Минимальные гарантии покрытия страхуют от ситуации когда примеры полностью вытесняют структурный контекст — в контексте всегда будут таблицы для генерации.

**Граф внешних ключей с алгоритмом Дейкстры.** Семантический поиск решает задачу релевантности, но не связности: найденные таблицы не обязательно образуют корректный JOIN-путь. Между ними могут быть промежуточные таблицы которые ретривал не нашёл, потому что в вопросе о них ничего нет. Без структурного знания модель либо галлюцинирует соединение напрямую, либо теряет связь между таблицами.

Граф знает о схеме то, чего не знает ретривал — какие таблицы являются связующими звеньями и как именно они соединяются. На первом шаге FK-расширение автоматически добавляет промежуточные таблицы в контекст. На втором алгоритм Дейкстры строит оптимальный JOIN-путь: весом ребра является удалённость узлов от центра графа — центральные связующие таблицы предпочтительнее периферийных.

**LLM-ассесор (LLM-as-a-Judge).** Для оценки качества сгенерированного SQL реализована роль судьи — отдельное обращение к модели с калиброванным промптом. Судья оценивает три критерия: соответствие намерению пользователя, корректность привязки к схеме и правильность проекции. Промпт содержит явные правила что не является ошибкой — порядок столбцов в ON, наличие алиасов, точка с запятой — и калиброванную шкалу score от 0 до 1. При отрицательном вердикте система передаёт описание ошибки обратно в генератор и запускает повторную генерацию с явным указанием что именно исправить. В текущей версии петля коррекции реализована, но отключена — это ближайшее направление развития.

## Масштабирование и развитие

Текущая реализация предполагает ряд ограничений на схему базы данных. Граф должен быть связным: между любыми двумя таблицами должен существовать путь через внешние ключи — изолированные таблицы не могут быть включены в JOIN-цепочку алгоритмом Дейкстры. Схема предполагается ациклической и простой: циклические зависимости и петли приводят к неоднозначности при построении пути. Retrieval-тексты, граф и значения отдалённости от центра написаны вручную — система не масштабируется на произвольную схему без ручной подготовки метаданных.

Направления развития выстраиваются в две группы.

**Качество ретривала и точность генерации.** После первичного семантического поиска можно добавить BERT-like Cross-Encoder для переранжирования результатов — он точнее оценивает релевантность пары запрос-документ и отсеивает шум до передачи контекста модели. Логирование запросов и ответов на длительном периоде использования формирует корпус реальных примеров для точного поиска по истории — система со временем становится умнее на собственном опыте. Автоматизация построения retrieval-текстов по образцу уже описанных таблиц снимает ограничение на масштабируемость и открывает применение к произвольным схемам без ручной разметки.

**Агентный подход через протокол MCP.** Переход от жёсткого пайплайна к агентной (ReAct) архитектуре с инструментами через MCP — наиболее принципиальное изменение. Модель сама решает когда обратиться к ретривалу, когда к графу, когда верифицировать SQL на реальной базе. Цикл исправлений при провале выполнения запроса становится естественной частью работы агента, а не отдельным компонентом. Диалоговый режим позволяет уточнять детали запроса у пользователя в случае неполноты информации — вместо генерации по неполному контексту система задаёт уточняющий вопрос. Агент с памятью удерживает контекст между запросами одной сессии, что особенно важно для многошаговых аналитических задач.